從提示工程到技能封裝

探索 Claude Skills 的架構與應用

演講者： Frank
2026/03/24
國立台北科技大學　資訊工程系
指導教授：鄭有進、謝金雲

Outline

1. **什麼是 Skill ？**
   定義、結構與核心設計原則
2. **Skills + MCP**
   知識層與連接層的協作關係
3. **規劃與設計**
   使用案例、 YAML Frontmatter 、指令撰寫
4. **測試與迭代**
   成功標準、測試方法、 Skill-Creator
5. **發布與設計模式**
   發布方式、五大模式、問題排除

什麼是 Skill ？

一組打包成資料夾的指令集
教導 Claude 處理特定任務或工作流程

教 Claude 一次，之後每次對話都能受益

適用場景：可重複的工作流程

- 從規格生成前端設計
- 以一致方法進行研究
- 建立遵循團隊風格的文件
- 協調多步驟流程

本指南的兩條路徑

獨立 Skill 開發者
聚焦 Fundamentals + Planning (Cat. 1-2)

MCP 整合開發者
聚焦 Skills + MCP 段落 (Cat. 3)

漸進式揭露（Progressive Disclosure）

Layer 1
YAML Frontmatter
永遠載入，判斷何時使用

▼

Layer 2
SKILL.md 主體
相關時載入，完整指令

▼

Layer 3
references/ 連結檔
按需載入，深度參考

核心效益

最小化 Token 使用量
同時保持專業能力

只在需要時載入需要的資訊

其他設計原則

- 可組合性：多個 Skill 可同時協同運作
- 可攜性： Claude.ai / Code / API 運作一致

Skills + MCP ：廚房比喻

MCP = 專業廚房（連接層）

- 連接 Claude 到服務（ Notion, Linear... ）
- 提供即時資料存取和工具調用

回答： Claude 能做什麼？

Skills = 食譜（知識層）

- 教導 Claude 如何使用服務
- 捕捉工作流程和最佳實踐

回答： Claude 該怎麼做？

沒有 Skills 時

- 使用者不知道下一步
- 每次對話從零開始
- 結果不一致，用戶歸咎於連接器

有 Skills 時

- 預建工作流程自動啟動
- 一致且可靠的工具使用
- 降低整合的學習曲線

Category 1 ：文件與資產建立

建立一致、高品質的輸出（文件、簡報、程式碼）

範例： frontend-design skill

「Create distinctive, production-grade frontend interfaces with high design quality.」

關鍵技巧

風格指南

嵌入品牌標準

範本結構

確保輸出一致性

品質檢查清單

完成前確認品質

無需外部工具

使用 Claude 內建能力

Category 2 & 3 ：工作流程自動化 & MCP 增強

Cat 2 ：工作流程自動化

多步驟流程，需要一致方法論

範例： skill-creator skill

技巧：

- 驗證關卡的逐步工作流程
- 迭代精煉循環

Cat 3 ： MCP 增強

為 MCP 工具加上工作流程引導

範例： sentry-code-review skill

技巧：

- 多 MCP 協調、領域知識嵌入
- 錯誤處理

使用案例範本（ Sprint Planning ）

Trigger: 「help me plan this sprint」 ▸ Step 1: 取得專案狀態 ▸ Step 2: 分析產能 ▸ Step 3: 建議優先順序 ▸ Step 4: 建立任務 ▸ Result: 完整的 Sprint 計畫

Skill 的檔案結構與命名規則

資料夾結構

```
your-skill-name/
  SKILL.md        （必要）
  scripts/        （選用）
  references/     （選用）
  assets/         （選用）
```

關鍵規則

SKILL.md 大小寫敏感
只接受 SKILL.md 這個拼法

資料夾用 kebab-case
notion-project-setup ✅
Notion Project Setup ❌
notion_project_setup ❌

不放 README.md
文件放 SKILL.md 或 references/

安全限制： 禁止 XML 標籤（< >）/ 名稱不可含「claude」或「anthropic」（保留字）

原因：Frontmatter 出現在 Claude 的 system prompt 中，惡意內容可能注入指令

YAML Frontmatter

最小必要格式

```
---
name: your-skill-name
description: What it does.
  Use when user asks to
  [specific phrases].
---
```

選用欄位

license:
MIT, Apache-2.0 等

compatibility:
環境需求（1-500 字元）

allowed-tools:
限制工具存取

metadata:
author, version, mcp-server

欄位要求摘要

欄位	規則
name	kebab-case，無空格或大寫，應與資料夾名稱一致
description	必須包含「做什麼」+「何時用」，上限 1024 字元，禁 XML 標籤

Description ： Skill 成敗的關鍵

✅ 好的 Description

具體說明「做什麼」 + 包含「何時使用」的觸發詞 + 提及相關檔案類型

「Manages Linear workflows including sprint planning. Use when user mentions sprint, Linear tasks, or project planning.」

❌ 壞的 Description

問題	範例	原因
太模糊	「Helps with projects」	Claude 幾乎不會觸發
缺觸發條件	「Creates documentation systems」	沒說何時該用
太技術化	「Implements entity model...」	用戶不會這樣說話

核心原則：使用者怎麼說，description 就怎麼寫

撰寫 SKILL.md 指令的最佳實踐

具體且可操作

✅

```
Run python scripts/validate.py
--input {filename}
```

❌

```
Validate the data before
proceeding
```

善用漸進式揭露

✅

```
核心指令放 SKILL.md
詳細文件移 references/
```

❌

```
所有內容擠在
SKILL.md 裡
```

包含錯誤處理與範例

✅

```
MCP 連線失敗時：
1. 確認服器狀態 ...
```

❌

```
(沒有錯誤處理)
```

進階技巧：用腳本做確定性驗證，而非語言指令。程式碼是確定性的，語言解讀不是。

定義成功標準

量化指標

90% 相關查詢觸發率

X 工具呼叫內完成

0 失敗的 API 呼叫

質化指標

- 用戶不需提示下一步
- 工作流程無需修正即完成
- 各工作階段結果一致
- 新用戶能否首次就完成？

效能對比範例（文件提供的參考基準）

	Without Skill	With Skill
來回對話	15 次	2 個澄清問題
失敗 API 呼叫	3 次	0 次
Token 消耗	12,000	6,000

測試的三個面向

1. 觸發測試

Skill 是否在正確時機載入？

✅ 明確任務觸發
✅ 改述請求也觸發
❌ 無關主題不觸發

2. 功能測試

輸出正確？ API 成功？錯誤處理？

驗證輸出結果
確認 API 呼叫成功
測試邊界情況

3. 效能比較

證明 Skill 確實改善了結果

對比訊息來回次數
比較失敗呼叫數量
比較 Token 消耗

測試方法

手動測試
Claude.ai 上直接跑

腳本測試
Claude Code 自動化

程式化測試
Skills API 評估套件

Skill-Creator 與迭代改進

Skill-Creator 工具

- 從自然語言描述生成 Skill
- 產生正確格式的 SKILL.md
- 審查並建議改善

使用：
「Use skill-creator to help me build a skill for [用途]」

迭代信號與對策

觸發不足	加更多觸發詞 / 技術用語
觸發過度	加負面觸發條件 / 縮小範圍
執行不一致	改善指令精確度 / 錯誤處理

Skills 是活文件，透過測試和使用者回饋不斷迭代改善

Pro Tip：先針對單一困難任務不斷迭代直到成功，再把成功的方法萃取成 Skill，而不是一開始就做廣泛測試。

發布與分享

個人

下載資料夾 → 壓縮 → 上傳 Claude.ai
或放入 Claude Code 目錄

組織

管理員在工作區部署
自動更新、集中管理

API

/v1/skills 端點
Messages API 整合
Agent SDK 支援

開放標準：如同 MCP ，Skill 可跨工具和平台使用

定位原則：聚焦成果而非功能

✅「幾秒內建立完整工作區，取代 30 分鐘手動設定」
❌「包含 YAML frontmatter 和 Markdown 指令的資料夾」

設計模式框架： Problem-first vs. Tool-first

Home Depot 比喻
（類似台灣的特力屋）你可能帶著問題進去，「我要修廚房櫃櫥」；或者拿起一把新電鑽，問怎麼用它。

Problem-first

用戶描述想要的結果

「幫我建立一個專案工作區」

Skill 負責編排正確的
MCP 呼叫順序

Tool-first

用戶已有 MCP 存取

「我連接了 Notion MCP 」

Skill 教導 Claude 最佳的
工作流程和實踐

Pattern 1 & 2 ：循序式工作流程 & 多 MCP 協調

Pattern 1 ：循序式工作流程

特定順序的多步驟流程

1. 建帳號
2. 設定付款
3. 建訂閱
4. 發歡迎信

Pattern 2 ：多 MCP 協調

跨多個服務的流程

1. Figma MCP → 匯出設計
2. Drive MCP → 儲存資產
3. Linear MCP → 建立任務
4. Slack MCP → 發送通知

Pattern 1 關鍵技巧

- 明確的步驟順序與相依性
- 每階段驗證 + 失敗時回滾

Pattern 2 關鍵技巧

- 清楚的階段分離
- MCP 之間的資料傳遞

Pattern 3-5 ：迭代精煉 / 上下文感知 / 領域智慧

Pattern 3
迭代精煉

輸出品質因反複修改而改善

範例：報告生成

關鍵技巧：

明確品質標準
知道何時停止迭代

Pattern 4
上下文感知選擇

相同目標，依情境選不同工具

範例：檔案儲存

關鍵技巧：

清楚的決策樹
透明的選擇說明

Pattern 5
領域專業智慧

嵌入超越工具存取的專業知識

範例：付款合規檢查

關鍵技巧：

領域專業嵌入邏輯
合規優先於執行

常見問題排除

上傳失敗

SKILL.md 拼寫不對（大小寫敏感）
YAML 缺少 --- 分隔符號
名稱沒用 kebab-case

觸發問題

不足 → 加觸發詞到 description
過度 → 加負面觸發、縮小範圍
除錯：問 Claude 「When would you use [skill]?」

指令未遵循

太冗長 → 精簡、條列式
關鍵指令被埋 → 放最上方
語言模糊 → 用腳本驗證

效能問題

SKILL.md < 5,000 字
詳細文件移 references/
避免同時啟用 >20 個 Skill

Quick Checklist & 資源

開始前

辨識 2-3 使用案例
規劃資料夾結構

開發中

kebab-case 命名
SKILL.md + 正確 YAML
指令清楚可操作

上傳前

觸發 + 功能測試通過
壓縮為 .zip

上傳後

真實對話測試
監控觸發、收集回饋
持續迭代

資源

skill-creator ：內建工具，快速生成和審查 Skill
github.com/anthropics/skills ：公開範例倉庫
支援： Claude Developers Discord / GitHub Issues

結論

- Skill 將「一次性的提示工程」封裝成「可重複使用的知識資產」
- 三層漸進式揭露：平衡 Token 效率與專業深度
- MCP + Skills = 連接能力 + 工作流程知識 → 完整的 AI 解決方案
- Description 是成敗關鍵 — 決定 Skill 何時被觸發
- 持續迭代：Skill 是活文件，透過測試與回饋不斷改善

從今天開始：用 skill-creator 花 15-30 分鐘建立你的第一個 Skill

Thanks for Listening

有什麼問題歡迎提問